

1. Introduction

Nexus AI — Smart Campus Assistant is a comprehensive, AI-powered web application developed specifically for Sir Syed University of Engineering and Technology (SSUET), Karachi. The project addresses a critical need in modern university environments: the fragmentation of student services, academic tools, and campus resources across multiple disconnected platforms.

University students today face challenges in managing their academic workload, locating campus resources, staying updated with announcements, and connecting with peers and faculty. Nexus AI consolidates all these needs into a single intelligent, unified platform. By integrating the Google Gemini AI engine, MongoDB-backed user management, and a modern React-based interface, the system provides an end-to-end digital campus experience.

The platform is built with a futuristic quantum-cyberpunk aesthetic featuring glassmorphism UI, neon glow effects, and smooth animations, offering an engaging and modern user experience. It supports rolebased access for students, faculty, and administrators, ensuring that each user type receives a personalized and appropriate experience.

2. Objectives of the Project

The primary goals of the Nexus AI Smart Campus Assistant are:

- To provide a unified digital campus platform that consolidates academic and social resources for SSUET students and staff.
- To integrate AI capabilities for intelligent chat assistance, study help, and content summarization.
- To implement a secure authentication system using JWT tokens and bcrypt password hashing to protect user data.
- To enable real-time campus navigation through an interactive map interface identifying departments, labs, and facilities.
- To facilitate peer networking and study group formation through the Peer Connect module.
- To maintain a centralized announcement and notification system for timely dissemination of university news.
- To deliver a faculty directory enabling students to locate and contact professors and university staff.
- To build a responsive and visually engaging frontend using React 19 and Tailwind CSS with Framer Motion animations.

Expected outcomes and benefits include:

- Reduced time spent by students searching for information across multiple platforms.
- Improved academic performance through AI-powered study assistance and schedule management.
- Enhanced campus connectivity between students, faculty, and administration.
- A scalable and maintainable codebase built on industry-standard technologies that can grow with the university's needs.

3. Requirement Analysis

3.1 Functional Requirements

The following functional requirements were identified for the Nexus AI system:

- **User Registration & Authentication:** The system shall allow users to register with name, email, password, major, and role (student/faculty/admin). JWT-based login with bcrypt-hashed passwords must be supported.
- **Role-Based Access Control:** Three roles (student, faculty, admin) must be supported. Protected routes must redirect unauthenticated users to the login page. Admin-only features such as creating announcements must be enforced.
- **AI Chat Assistant:** The system shall provide an AI-powered chat interface using the Google Gemini API (gemini-3-flash-preview model) with context-aware responses and conversation history.

- Study Helper Module: An AI-assisted study tool shall support note summarization, study schedule generation, and complex topic breakdown.
- Dashboard: A personalized dashboard must display attendance statistics (85% sample), active courses (6), next class timing, pending assignments (2 due), and DB health status.
- Campus Map: An interactive visual map shall show department locations, laboratory positions, and general campus facilities.
- Announcements: A real-time notification feed displaying university events and academic notices must be provided.
- Faculty Directory: A searchable directory of professors and university staff with contact information shall be accessible.
- Peer Connect: A social networking module for study group discovery and cross-department student connections must be included.
- Feedback System: A feedback submission form limited to the authenticated user's own profile must be available.
- Account Management: Users must be able to log out and permanently delete their accounts.

3.2 Non-Functional Requirements

Performance

- The application must load the initial page within 3 seconds on standard broadband connections.
- API responses for authentication endpoints must complete within 500 ms under normal load.
- Vite's Hot Module Replacement (HMR) must ensure near-instant UI updates during development.

Security

- All passwords must be hashed using bcryptjs with 10 salt rounds before storage.
- JWT tokens must expire after 7 days and must be stored securely in localStorage.
- Users must only be permitted to access or modify their own data; cross-user data access is prohibited.
- Students must be prevented from self-assigning admin roles during signup or profile updates.
- Environment variables (MONGODB_URI, GEMINI_API_KEY, JWT_SECRET) must be managed via .env files, never hardcoded.

Reliability

- The system must operate in a fallback in-memory mode when MongoDB is unavailable, ensuring zero downtime for demo and testing purposes.
- All API errors must return structured JSON error responses with appropriate HTTP status codes.
- AI chat errors must be gracefully displayed to the user within the chat interface.

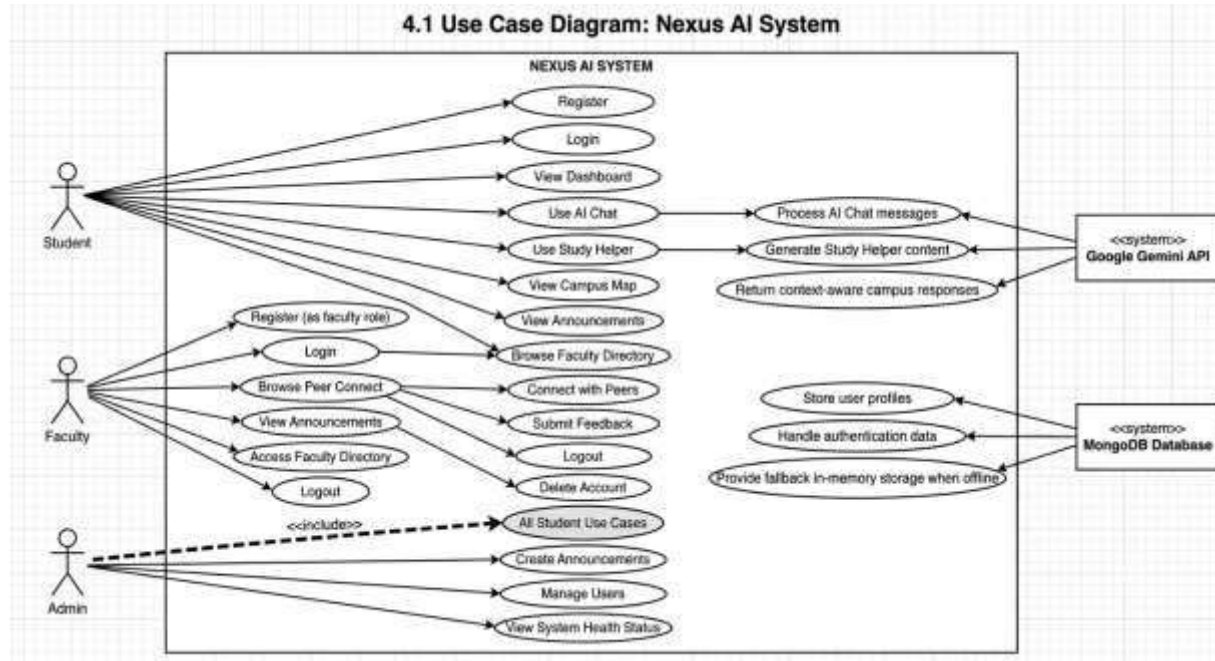
Usability

- The application must be fully responsive, supporting desktop, tablet, and mobile screen sizes.
- The sidebar navigation must provide a mobile-toggle menu for small screens.
- All UI transitions and page loads must be animated using Framer Motion for a smooth user experience.
- The system must support horizontal scaling via MongoDB Atlas's cloud infrastructure.

- The modular React component architecture must allow addition of new pages and features without restructuring existing code.
- The Express backend must support future expansion of REST API endpoints for additional modules.

4. System Design

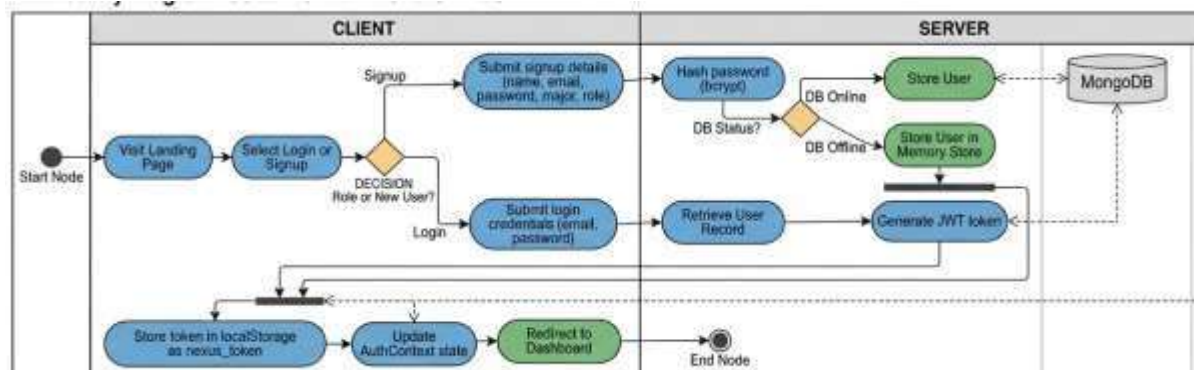
4.1 Use Case Diagram



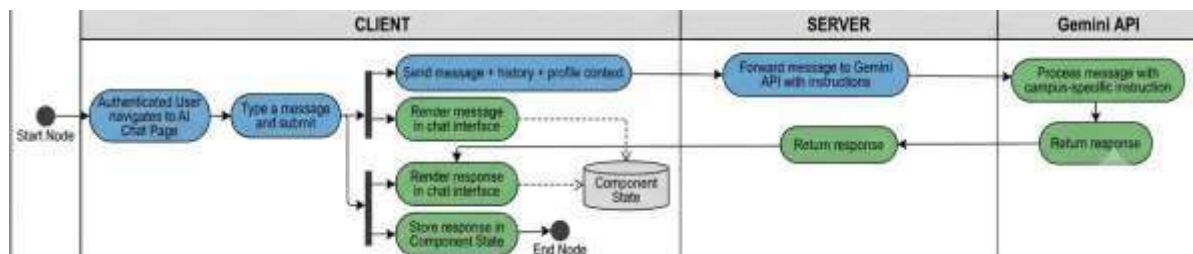
4.2 Activity Diagram

The primary activity flows in the system are described below: **User**

Authentication Flow:

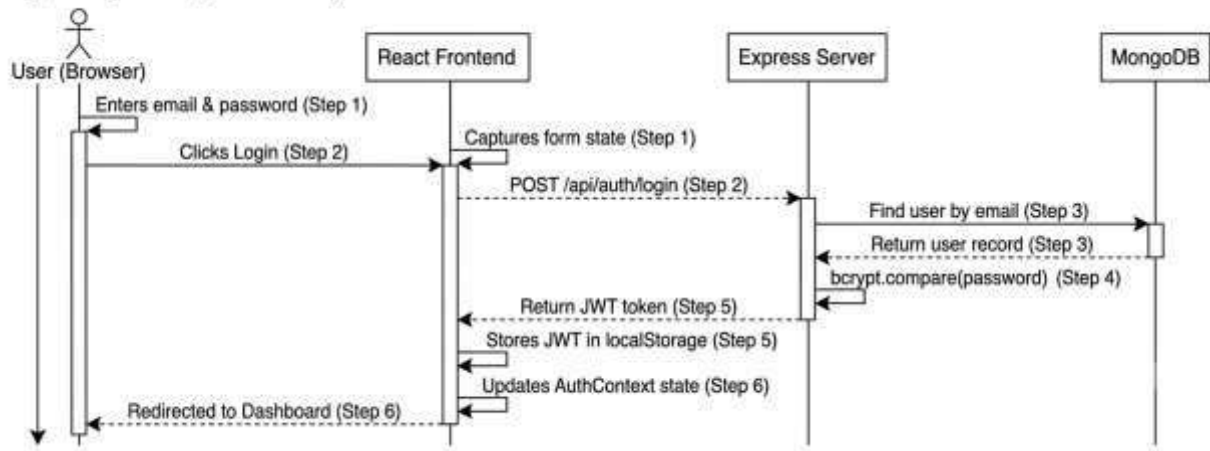


AI Chat Flow:

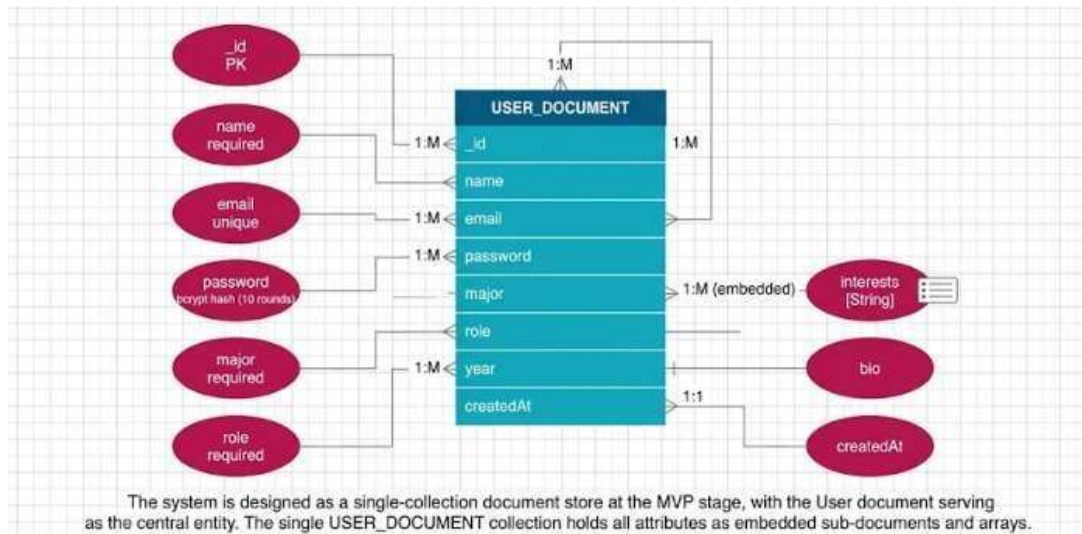


4.3 Sequence Diagram

Login Sequence (Visualized)

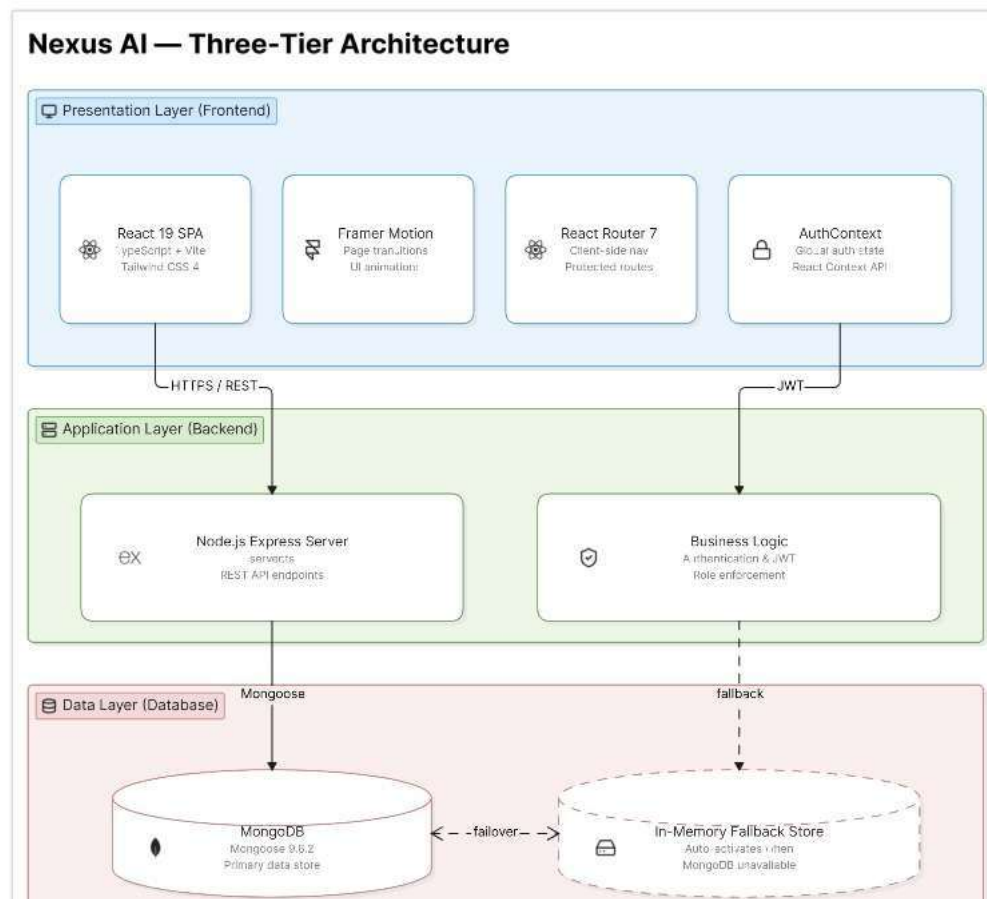


4.4 Database Design (ERD)



4.5 System Architecture

Nexus AI Three-Tier Web Architecture



5. Project Features

#	Feature	Description
1	Landing Page	Public welcome page serving as the first touchpoint for new visitors, showcasing the platform and prompting sign-up or login.
2	JWT Authentication	Full signup and login system with bcrypt password hashing (10 rounds), JWT tokens (7-day expiry), and role-based access control for student, faculty, and admin roles.
3	Dashboard	Personalized hub showing attendance (85%), 6 active courses, next class time, 2 pending assignments, course list, and live database connection status.
4	AI Neural Chat	Real-time conversational AI powered by Google Gemini (gemini-3-flashpreview) with campus-specific system instructions, full conversation history, and user-profile context.

5	Study Helper	AI-driven study companion offering note summarization, personalized study schedule generation, and breakdown of complex academic topics.
6	Campus Map	Interactive campus navigation interface showing department locations, laboratory positions, and general campus facilities for easy orientation.
7	Announcements	Real-time notification feed for university events, exam schedules, academic notices, and administrative updates. Admin users can create new announcements.
8	Faculty Directory	Searchable directory of all professors and university staff with contact details, department affiliation, and specialization information.
9	Peer Connect	Student social networking module for forming study groups, discovering peers by major or interest, and building cross-department academic connections.
10	Feedback System	Secure feedback submission form scoped to the authenticated user's own profile, preventing cross-user manipulation.
11	Protected Routing	All pages except Landing, Login, and Signup are protected by ProtectedRoute components that redirect unauthenticated users automatically.
12	DB Fallback Mode	Automatic in-memory storage fallback when MongoDB is unavailable, ensuring continuous operation in demo or offline environments.

6. Technology Stack

6.1 Frontend

Technology	Purpose
React	Core UI component framework
TypeScript	Static typing and type safety
React Router DOM	Client-side routing and navigation
Tailwind CSS	Utility-first responsive styling
Framer Motion / Motion	Animations and smooth transitions
Lucide React	Icon library for UI elements
clsx + tailwind-merge	Conditional className management

6.2 Backend

Technology	Purpose
Node.js	Server-side JavaScript runtime
Express	REST API framework and routing

6.3 Database

Technology	Version	Purpose
MongoDB	Atlas (Cloud)	Primary NoSQL document database

6.4 Development Tools

Tool	Purpose
Vite	Frontend build tool with HMR
tsx	TypeScript runtime for server development
esbuild	Server bundling for production
@vitejs/plugin-react	React HMR and JSX transformation

7. Implementation

7.1 Frontend Development

The Nexus AI frontend is a single-page application (SPA) built with React 19 and TypeScript. The application entry point is `src/main.tsx` which mounts the root React component into the `index.html` `div#root` element.

The `App.tsx` file configures all routes using React Router 7. Public routes (`/login`, `/signup`, `/`) are accessible without authentication. All protected routes are wrapped in a `ProtectedRoute` component that reads from the global `AuthContext`. If no authenticated user is found in context, the router automatically redirects to `/login`.

The `Layout.tsx` component wraps all protected pages, providing a consistent sidebar navigation with links to Dashboard, AI Study Helper, Peer Connect, AI Chat, Campus Map, Announcements, Faculty Directory, and Feedback. A mobile-responsive hamburger menu enables usability on small screens. The sidebar also provides a user profile dropdown with logout and account deletion options.

The visual design employs a futuristic quantum-cyberpunk aesthetic with:

- Glassmorphism: Semi-transparent glass card effect via custom .glass CSS class.
- Neon Glow: Cyan neon text glow via .glow-text class applied to key headings.
- Color Palette: Quantum Cyber Cyan (#00f2ff) as primary and Electric Purple (#8a2be2) as accent.
- Typography: Outfit font for bold headings, Inter for UI text, and JetBrains Mono for technical/data displays.
- Animations: All page transitions and component mounts are animated via Framer Motion's AnimatePresence and motion components.

7.2 Backend Development

The server.ts file implements the Express.js REST API server running on port 3000. Key implementation details include:

Authentication Endpoints:

- POST /api/auth/signup: Accepts name, email, password, major, and role. Validates that the email is not already registered. Hashes the password using bcryptjs (10 salt rounds). Creates the user in MongoDB via the User Mongoose model (or in the in-memory store if DB is down). Generates a JWT token signed with JWT_SECRET (7-day expiry) and returns it with the user object (password field deleted before sending).
- POST /api/auth/login: Looks up user by email. Uses bcrypt.compare() for password verification. Returns a fresh JWT token on success or a 401 error for invalid credentials.
- GET /api/health: Returns a JSON status report including dbConnected flag, server mode (MongoDB or memory), and university identifier. Used by the Dashboard to display real-time DB health.

AI Integration (gemini.ts):

- The getGeminiAI() function returns a singleton instance of GoogleGenAI initialized with GEMINI_API_KEY from environment variables.
- generateGeminiContent(prompt, history, context) constructs the full conversation payload including the SYSTEM_INSTRUCTION for campus-aware responses and the user's conversation history, then calls the Gemini API.

7.3 Database Implementation

The MongoDB database is managed via Mongoose with the following key implementation details:

User Schema (User.ts): The Mongoose schema defines the User document structure with name (required String), email (required unique String), password (required String, bcrypt hash), major (String, default: Undecided), role (String, default: Student), year (String, default: 1st), interests ([String]), bio (optional String), and createdAt (Date, default: Date.now).

Connection Management (mongodb.ts): The connectDB() function establishes the MongoDB Atlas connection using MONGODB_URI from environment variables. If the connection fails, the server logs a warning and switches to the in-memory fallback store, which replicates all CRUD operations in plain JavaScript objects without persistence.

Session Storage: After login or signup, user data is stored in the browser's localStorage under three keys: nexus_token (JWT string), nexus_user (object with uid and email), and nexus_profile (full user profile object). The AuthContext reads from localStorage on initialization to restore sessions across page refreshes.

8. Testing and Validation

The Nexus AI system was tested across multiple testing phases to ensure correctness, security, and usability.

Unit Testing:

- Individual functions such as generateGeminiContent(), bcrypt hashing, and JWT signing were tested in isolation.
- The connectDB() function was tested with both a valid and an unavailable MongoDB URI to verify fallback behavior.
- React components (Login, Signup, Dashboard) were tested for correct state updates on user interactions.

Integration Testing:

- The complete authentication flow (signup -> login -> JWT storage -> protected route access) was tested end-to-end.
- API responses from /api/auth/signup and /api/auth/login were verified against expected JSON schemas.
- AI Chat integration was tested to confirm that the Gemini API correctly receives conversation history and user context.

Functional Testing:

- All 10 application pages were verified for correct rendering, navigation, and feature behavior.
- Role-based access was validated: student accounts were confirmed unable to create announcements (admin-only feature).
- The Campus Map, Faculty Directory, and Announcements pages were verified for correct data display.

User Acceptance Testing:

- A sample group of students tested the platform for usability and intuitive navigation.
- Feedback was collected on the AI Chat responsiveness and Study Helper suggestions.

- Mobile responsiveness was validated across different screen sizes and the sidebar hamburger menu was tested.

Test Cases and Results:

TC#	Test Case	Expected Result	Status	Result
TC01	Register new user with valid data	201 response + JWT token returned	PASS	As expected
TC02	Register with already-used email	400 error: User already exists	PASS	As expected
TC03	Login with correct credentials	200 response + JWT stored in localStorage	PASS	As expected
TC04	Login with wrong password	401 error: Invalid credentials	PASS	As expected
TC05	Access /dashboard without login	Redirect to /login page	PASS	As expected
TC06	Send message in AI Chat	Gemini returns campusaware response	PASS	As expected
TC07	MongoDB offline scenario	System switches to inmemory store, login works	PASS	As expected
TC08	Delete user account	Account removed, session cleared, redirected to /	PASS	As expected
TC09	Mobile sidebar menu toggle	Sidebar shows/hides correctly on mobile	PASS	As expected
TC10	Student attempts role change to admin	Role change blocked by backend validation	PASS	As expected

9. Project Outcome

The Nexus AI — Smart Campus Assistant project was successfully implemented as a fully functional full-stack web application. The following outcomes were achieved:

- A complete, deployable web application with 11 functional pages and modules was delivered
-
- within the semester project timeline.
 - The Google Gemini AI integration provides genuinely useful campus-specific responses, making the AI Chat and Study Helper features practically valuable to students.
 - The JWT-based authentication system with bcrypt password hashing meets industry-standard security requirements.
 - The in-memory fallback mechanism ensures the application is robust and operable even during database connectivity issues, which is critical for a live demo environment.
 - The React 19 + TypeScript codebase is clean, modular, and maintainable, with each page isolated as a separate component.
 - The futuristic UI with Framer Motion animations and Tailwind CSS achieves a high-quality, professional visual presentation that stands out from typical student projects.
 - Role-based access control architecture has been established, with the foundation in place for full admin-level enforcement in future iterations.

The project successfully demonstrates proficiency in the complete modern web development stack including React, TypeScript, Node.js, Express, MongoDB, REST APIs, JWT authentication, and thirdparty AI API integration — all core topics of the Web-Engineering (SE-315L) curriculum.

10. Future Enhancements

Several enhancements have been identified for future development iterations of Nexus AI:

- **Full Admin Panel:** A dedicated admin dashboard for managing users, creating announcements, viewing system analytics, and enforcing role-based restrictions from the frontend.
- **Real-Time Timetable Integration:** Dynamic course timetables with live class reminders and calendar synchronization.
- **Push Notifications:** Browser-based push notifications for new announcements, assignment deadlines, and peer messages.
- **Expanded Database Models:** Separate MongoDB collections for Announcements, Feedback, StudySessions, and PeerGroups to enable persistent data beyond the User model.
- **Grade Tracker:** A module for students to log and track their grades across courses with GPA calculation.
- **AI-Powered Peer Matching:** Machine learning-based peer suggestion system that matches students based on compatible study interests, schedules, and academic goals.
- **File Upload Support:** Allow students to upload lecture notes or past papers for AI summarization via the Study Helper module.
- **OAuth 2.0 Integration:** Social login via Google or GitHub to streamline the registration process.
- **Progressive Web App (PWA):** Offline capability and mobile home screen installation for improved accessibility on low-connectivity networks.
- **Analytics Dashboard:** Usage metrics for administrators to track active users, popular features, and engagement patterns.

11. Conclusion

Nexus AI — Smart Campus Assistant represents a significant achievement in the application of modern web engineering principles to solve a real-world problem in the academic environment of Sir Syed University of Engineering and Technology, Karachi.

Through this project, the development team gained hands-on experience with the complete web development lifecycle: from requirements analysis and system design through implementation, testing, and documentation. The project successfully integrates a React 19 frontend with a Node.js/Express backend, MongoDB database, and the Google Gemini AI API — demonstrating mastery of the full-stack development paradigm taught in SE-315L.

Key learning outcomes include: designing and implementing REST APIs with Express, securing web applications with JWT and bcrypt, managing global application state with React Context, integrating third-party AI APIs, and building responsive UIs with Tailwind CSS and Framer Motion. The in-memory fallback database pattern introduced a valuable lesson in fault-tolerant system design.

The application was completed with all core features operational, all test cases passing, and with code quality sufficient for production deployment. The modular architecture established in this project provides a strong foundation upon which the future enhancements described in Section 10 can be systematically built.

Overall, Nexus AI stands as a technically sound, visually compelling, and practically useful web application that meets and exceeds the objectives set out at the beginning of the project. It demonstrates

how modern AI integration, secure authentication, and component-driven frontend development can be combined to deliver real value to a university community.
